Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Домашняя работа по дисциплине
«Инженерия данных»

«MapReduce и DuckDB поверх объектного хранилища RustFS: навыки и зарплаты в вакансиях Go»

Вариант: Go

Лектор: _____

Выполнил работу
студент:

Баранов Владимир Александрович, 407921, J3313

Санкт-Петербург, 2026

1. Данные и вариант

Данные те же, что в ДЗ №2: `it_vacancies_2024.csv`, вакансии hh.ru за декабрь 2023 — сентябрь 2024, 847 619 384 байт, 229 388 записей, 23 столбца. Переводов строк внутри полей нет, поэтому файл можно резать на сплиты по байтам.

Вакансия относится к сегменту Go, если в её названии есть отдельное слово Go или Golang (регистр не важен, граница слова — любой символ, кроме латинских букв, цифр и подчёркивания). Отбор идёт по названию: навык Go встречается и в вакансиях Python- и PHP-разработчиков («будет плюсом»), а роль задаёт название. Вакансий в сегменте: 2 751, с заполненными навыками: 2 038. Найдено одно ложное срабатывание: 5 вакансий «LQA-инженер … Яндекс Go», где Go — название сервиса; зарплаты у них нет, на результаты они не влияют.

Код лежит в репозитории: движок MapReduce — `hw/mapreduce.py`, задачи — `hw/jobs.py`, правила отбора и нормализации — `hw/normalize.py`, SQL для DuckDB — `sql/`, запуск этапов — `scripts/`, тесты — `tests/`. Все выводы команд целиком лежат в `logs/`, ниже приведены выдержки. Весь прогон повторяется командой `bash scripts/run_all.sh`.

2. Этапы 1–2. RustFS и загрузка

RustFS 1.0.1 поднят в Docker Compose по официальному примеру `docker-compose-simple.yml` на четырёх папках-дисках (`docker-compose.yml`). В ДЗ №2 была ВМ с настоящими дисками, потому что проверялись отказы; здесь хранилище нужно только как источник данных. Все тома лежат на одном диске ВМ Docker Desktop, поэтому включён флаг `RUSTFS_UNSAFE_BYPASS_DISK_CHECK=true`, допустимый для учебного стенда. S3 API — на порту 9100, ключи — в `.env`. Состояние стенда (`scripts/stand_info.sh`):

```
$ docker version --format $'Docker: \320\272\320\273\320\270\320\265\320\275\321\202 {{.Client.Version}},
↪  \321\201\320\265\321\200\320\262\320\265\321\200 {{.Server.Version}}'
Docker: клиент 28.2.2, сервер 29.4.3

$ docker compose version
Docker Compose version v5.1.4

$ docker compose ps --format table\ \{\{.Name\}\}\\t\{\{.Image\}\}\\t\{\{.Status\}\}\\t\{\{.Ports\}\}
NAME         IMAGE                 STATUS                  PORTS
hw3-rustfs   rustfs/rustfs:1.0.1   Up 2 hours (healthy)   0.0.0.0:9100->9000/tcp, [::]:9100->9000/tcp,
↪  0.0.0.0:9101->9001/tcp, [::]:9101->9001/tcp

$ docker exec hw3-rustfs rustfs --version
rustfs 1.0.1
build time   : 2026-10-03 03:14:05 +00:00
build profile: release
build os     : linux-aarch64
rust version : rustc 1.99.0 (b940084d7 2026-09-28)
rust channel : stable-aarch64-unknown-linux-gnu
git branch   :
git commit   : 6de965ae3c965a78ff819fbcd7acd4aa44177d92
git tag      : 1.0.1
git status   :


$ sh -c docker\ logs\ hw3-rustfs\ 2\>\&1\ \|\ head\ -3
Initializing data directories: /data/rustfs0 /data/rustfs1 /data/rustfs2 /data/rustfs3
Initializing log directory: /logs
Starting: /usr/bin/rustfs  /data/rustfs0 /data/rustfs1 /data/rustfs2 /data/rustfs3

$ curl -s -w \\nHTTP\ %\{http_code\}\\n http://localhost:9100/health/ready
{"status":"ok","service":"rustfs-endpoint","timestamp":"2026-10-19T13:25:01.394959633Z[Etc/Unknown]","version":"1.0.1"
↪  ,"ready":true,"details":{"storage":{"status":"connected","ready":true,"readinessScope":"read_quorum","source":"loc
↪  al_runtime","readQuorum":true,"writeQuorum":true,"unavailableDrives":[]},"iam":{"status":"connected","ready":true}
↪  ,"lock":{"status":"connected","ready":true},"poolMetadata":{"ready":true,"status":"writable"}},"degradedReasons":[
↪  ]}
HTTP 200
```

```
$ docker exec hw3-rustfs sh -c du\ -sh\ /data/rustfs\*
822M        /data/rustfs0
822M        /data/rustfs1
822M        /data/rustfs2
822M        /data/rustfs3

$ docker exec hw3-rustfs sh -c find\ /data/rustfs0/vacancies/raw\ -type\ f\ \|\ sort\ \|\ head\ -8\;\ echo\ ...\;\ find\
↪  /data/rustfs0/vacancies/raw\ -type\ f\ \|\ wc\ -l
/data/rustfs0/vacancies/raw/it_vacancies_2024.csv/c83913b2-6468-4d2d-9122-0166aaf25a0d/part.1
/data/rustfs0/vacancies/raw/it_vacancies_2024.csv/c83913b2-6468-4d2d-9122-0166aaf25a0d/part.1.meta
/data/rustfs0/vacancies/raw/it_vacancies_2024.csv/c83913b2-6468-4d2d-9122-0166aaf25a0d/part.10
/data/rustfs0/vacancies/raw/it_vacancies_2024.csv/c83913b2-6468-4d2d-9122-0166aaf25a0d/part.10.meta
/data/rustfs0/vacancies/raw/it_vacancies_2024.csv/c83913b2-6468-4d2d-9122-0166aaf25a0d/part.11
/data/rustfs0/vacancies/raw/it_vacancies_2024.csv/c83913b2-6468-4d2d-9122-0166aaf25a0d/part.11.meta
/data/rustfs0/vacancies/raw/it_vacancies_2024.csv/c83913b2-6468-4d2d-9122-0166aaf25a0d/part.12
/data/rustfs0/vacancies/raw/it_vacancies_2024.csv/c83913b2-6468-4d2d-9122-0166aaf25a0d/part.12.meta
...
103
```

`scripts/upload.py` загружает CSV частями по 16 МиБ и сверяет SHA-256, заново прочитав объект из RustFS:

```
RustFS: http://localhost:9100
Источник: it_vacancies_2024.csv (847,619,384 байт)
SHA-256 локально: 303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
Бакет vacancies уже есть
Загружено s3://vacancies/raw/it_vacancies_2024.csv за 4.3 с (187 МиБ/с)
HEAD: размер 847,619,384 байт, ETag "be9c1b4b3f57ff0e9771004c2a5fe78b-51"
SHA-256 из RustFS: 303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
Целостность: OK, хеши совпали

Объекты с префиксом raw/:
    847,619,384  raw/it_vacancies_2024.csv

Сколько этот объект занимает на каждой папке-диске RustFS (EC 2+2: 2 части данных + 2 чётности):
423846468       /data/rustfs0/vacancies/raw/it_vacancies_2024.csv
423846468       /data/rustfs1/vacancies/raw/it_vacancies_2024.csv
423846468       /data/rustfs2/vacancies/raw/it_vacancies_2024.csv
423846468       /data/rustfs3/vacancies/raw/it_vacancies_2024.csv
```

Объект занимает на каждой папке по 423 846 468 байт, вместе в 2,00 раза больше файла. Схему кодирования показывает `xl.meta` каждого диска (`scripts/ec_layout.py`): две части данных и две чётности.

```
Объект s3://vacancies/raw/it_vacancies_2024.csv
/data/rustfs0: шард 4 из 4 — чётность
/data/rustfs1: шард 1 из 4 — данные
/data/rustfs2: шард 2 из 4 — данные
/data/rustfs3: шард 3 из 4 — чётность
Схема: EcM = 2 (части данных), EcN = 2 (части чётности); блок 1,048,576 байт; частей multipart: 51; порядок дисков EcDist
↪  = [4, 1, 2, 3]
```

3. Этап 3. Подсчёт навыков двумя способами

Частота навыка, пары или тройки — число Go-вакансий, где встречается этот набор. Оба способа нормализуют навыки одинаково: разбивают по «;», обрезают пробелы и невидимые символы по краям (NBSP, zero-width space, BOM), приводят к нижнему регистру по одной букве, как `lower()` в DuckDB, склеивают 36 синонимов (`golang`→`go`, `k8s`→`kubernetes`, `apache kafka`→`kafka` и т. п., `config/skill_aliases.csv`) и убирают повторы. Без этого «Golang» и «Go» считались бы разными навыками, а «go + golang» стала бы второй по частоте парой.

Свой MapReduce (`hw/mapreduce.py`) устроен как в Hadoop. Объект режется на сплиты по 32 МиБ, и каждый map-таск читает свой сплит одним GET-запросом с заголовком `Range`. Строка принадлежит сплиту, в котором лежит её первый байт: таск пропускает начало чужой строки и дочитывает хвост своей последней. Map выдаёт пары `(комбинация, 1)`, combiner складывает их на месте, shuffle раскладывает ключи по 4 партициям по `crc32` в файлы, reduce

суммирует. Таски выполняются в пуле из 8 процессов, запущенных через `spawn`: при `fork` воркеры получили бы соединение boto3 родителя, и ответы на запросы перепутались бы. Перед разбором CSV строки проходят предфильтр по слову go/golang. Краевые случаи сплитов (сплит в 1 байт, граница на переводе строки, строка длиннее сплита, файл без последнего перевода строки) проверяют 209 тестов в `tests/`.

DuckDB читает тот же объект через расширение `httpfs`: одним чтением строит таблицы вакансий Go и пар «вакансия — навык» (`sql/go_segment.sql`), затем считает навыки, пары и тройки самосоединениями (`sql/skill_counts.sql`).

Выдержки из выводов `scripts.run_mapreduce` и `scripts.run_duckdb`:

```
Источник: s3://vacancies/raw/it_vacancies_2024.csv, вариант optimized
== MapReduce, задача skill_counts
сплитов 26, GET-запросов 51, прочитано 849,257,809 байт
записей 229,388, разобрано как CSV 8,720, битых строк 0
map выдал 161,469 пар (key, value), в shuffle ушло 105,324 ключей
уникальных ключей по размеру комбинации: k=1: 623, k=2: 7,124, k=3: 38,414
фазы, с: setup_s 0.23, map_s 6.47, reduce_s 0.24, total_s 6.95
профиль: wall_s 6.96, cpu_s 11.474, cpu_util 1.65, peak_rss_tree_mb 1015.0, peak_rss_main_mb 65.4, maxrss_main_mb 65.4,
↪  maxrss_child_mb 138.7, peak_procs 10

== MapReduce, задача salary_by_combination
сплитов 26, GET-запросов 51, прочитано 849,257,809 байт
записей 229,388, разобрано как CSV 8,720, битых строк 0
map выдал 58,989 пар (key, value), в shuffle ушло 44,790 ключей
уникальных ключей по размеру комбинации: k=0: 1, k=1: 296, k=2: 3,238, k=3: 16,563
фазы, с: setup_s 0.02, map_s 4.78, reduce_s 0.25, total_s 5.04
профиль: wall_s 5.056, cpu_s 9.458, cpu_util 1.87, peak_rss_tree_mb 1082.8, peak_rss_main_mb 62.6, maxrss_main_mb 65.4,
↪  maxrss_child_mb 156.0, peak_procs 10
Go-вакансии с зарплатой: 465 уникальных предложений от 258 работодателей, медиана 250,000 руб. на руки

Результаты в RustFS: s3://vacancies/results/mapreduce/combo_salary.csv, s3://vacancies/results/mapreduce/pairs.csv,
↪  s3://vacancies/results/mapreduce/skills.csv, s3://vacancies/results/mapreduce/triples.csv
```

```
DuckDB 1.5.6, потоков 12, вариант materialized
Источник: read_csv('s3://vacancies/raw/it_vacancies_2024.csv', header = true, all_varchar = true)
go_vacancies: 2,751 вакансий; go_skills: 11,540 строк (вакансий с навыками 2,038)
-- skills.csv: 623 строк
-- pairs.csv: 7,124 строк
-- triples.csv: 38,414 строк
фазы, с: load_s 4.457, total_s 4.492
профиль: wall_s 4.498, cpu_s 3.429, cpu_util 0.76, peak_rss_tree_mb 435.5, peak_rss_main_mb 435.5, maxrss_main_mb 435.5,
↪  maxrss_child_mb 0.0, peak_procs 1
```

MapReduce прочитал все 229 388 записей без битых строк; как CSV разобраны только 3,8 % строк, прошедших предфильтр. Различных навыков: 623, пар: 7 124, троек: 38 414.

Таблица 1 – Топ-10 навыков, пар и троек (число Go-вакансий; вакансий с навыками — 2 038)

№	навык	вак.	пара	вак.	тройка	вак.
1	go	1 911	go + postgresql	946	docker + go + postgresql	372
2	postgresql	990	docker + go	559	go + kafka + postgresql	355
3	docker	579	go + kafka	495	go + kubernetes + postgresql	250
4	kafka	511	go + sql	414	git + go + postgresql	240
5	sql	433	go + kubernetes	411	docker + go + kubernetes	222
6	kubernetes	424	go + linux	395	go + postgresql + sql	216
7	git	423	git + go	391	docker + git + go	207
8	linux	407	docker + postgresql	384	go + postgresql + rabbitmq	206
9	redis	317	kafka + postgresql	366	go + postgresql + redis	206
10	rabbitmq	312	go + redis	305	docker + go + kafka	173

Навык «go» указан в 93,8 % вакансий с навыками, поэтому частые пары и тройки почти все содержат его. Рядом с Go чаще всего стоят PostgreSQL, Docker, Kafka, SQL, Kubernetes, Git и Linux.

4. Этап 4. Верификация

`scripts/verify.py` читает результаты обоих способов из RustFS и сравнивает поэлементно: множества ключей, каждую частоту, сумму и SHA-256. Тем же скриптом сверяются медианы зарплат этапа 5 по всем 20 098 комбинациям.

```
файл              MapReduce   DuckDB только MR только DB разные числа
skills.csv             623      623       0       0         0   OK
                  сумма частот 11,540 = 11,540; SHA-256 84fbda904a01073e / 84fbda904a01073e
pairs.csv            7,124    7,124       0       0         0   OK
                  сумма частот 39,256 = 39,256; SHA-256 1f165b4be04d5005 / 1f165b4be04d5005
triples.csv         38,414   38,414       0       0         0   OK
                  сумма частот 110,673 = 110,673; SHA-256 9c91dd12b2f9e7ee / 9c91dd12b2f9e7ee
combo_salary.csv    20,098   20,098       0       0         0   OK
                  (предложений, работодателей, медиана) для всех Go-вакансий: MapReduce (465, 258, 250000.0), DuckDB
                  ↪  (465, 258, 250000.0)

ИТОГ: результаты полностью совпадают
```

Расхождений нет, хотя правила реализованы независимо: Python `csv`, `re` и `statistics.median` в MapReduce и CSV-ридер, RE2 и `median()` в DuckDB.

5. Этап 5. Навыки и зарплата

Методика (`sql/salary.sql`, `scripts/analyze.py`):

- берутся только рубли (без USD, EUR и KZT: 47 вакансий); точка — середина вилки или известная граница; сумма «до вычета налогов» умножается на 0,87; всё вне 10 тыс.–1,5 млн руб. считается выбросом;
- одна вакансия часто размещена в нескольких городах, поэтому копии с одинаковыми работодателем, названием, навыками и вилкой склеиваются в одно предложение: из 629 записей остаётся 465 предложений от 258 работодателей;
- влияние комбинации — разница медиан предложений с ней и остальных предложений со списком навыков; комбинация должна встретиться минимум в 15 предложениях от 5 работодателей, сам «go» в пары и тройки не входит;
- устойчивость проверяется перестановочным тестом (p, 5 000 перестановок) и той же разницей внутри одного уровня опыта, 3–6 лет.

Медиана по сегменту — 250 000 руб./мес. на руки (квартили 175 125 и 350 000). Сильнее всего зарплата зависит от опыта:

Таблица 2 – Зарплата по требуемому опыту, руб./мес. на руки

опыт	предложений	p25	медиана	p75
Нет опыта	23	48 925	91 350	225 000
От 1 года до 3 лет	157	120 000	180 000	250 000
От 3 до 6 лет	249	225 000	300 000	365 400
Более 6 лет	36	318 196	350 000	500 000

В верхнем квартиле зарплат в вакансии указано меньше навыков, чем в нижнем (4,8 против 7,6), и это сохраняется среди предложений с полной вилкой (4,9 против 8,6). Длинный перечень «Git, SQL, MySQL, Linux, REST, PHP» типичен для младших и смешанных вакансий. Правило «одна граница — это точка» завышает вилки «до X»: они составляют 65,3 % верхнего квартиля.

Таблица 3 – Пары навыков: сильнее всего повышают (сверху) и понижают (снизу) медиану; «Δ 3–6 лет» — разница при одинаковом опыте, прочерк — мало данных

комбинация	предл.	медиана с	медиана без	Δ, %	p	Δ 3–6 лет
clickhouse + kafka	15	300 000	250 000	+20,0	0,299	10 000
clickhouse + docker	18	300 000	250 000	+20,0	0,181	0
clickhouse + git	16	300 000	250 000	+20,0	0,204	0
clickhouse + postgresql	27	300 000	250 000	+20,0	0,176	18 625
ci/cd + postgresql	15	300 000	250 000	+20,0	0,299	0
postgresql + микросервисы	15	300 000	250 000	+20,0	0,299	15 000
mongodb + redis	17	300 000	250 000	+20,0	0,272	-25 000
kafka + redis	24	300 000	250 000	+20,0	0,142	58 000
git + mysql	34	127 750	250 000	-48,9	0,000	-50 000
rest + sql	20	137 500	250 000	-45,0	0,001	-66 250
python + sql	20	140 000	250 000	-44,0	0,001	-74 638
mysql + rest	17	147 900	250 000	-40,8	0,003	-66 250
git + python	23	150 000	250 000	-40,0	0,004	-89 025
git + php	19	150 000	250 000	-40,0	0,009	-50 000
linux + mysql	16	155 000	250 000	-38,0	0,017	-80 000
mysql + php	20	170 000	250 000	-32,0	0,019	-85 000

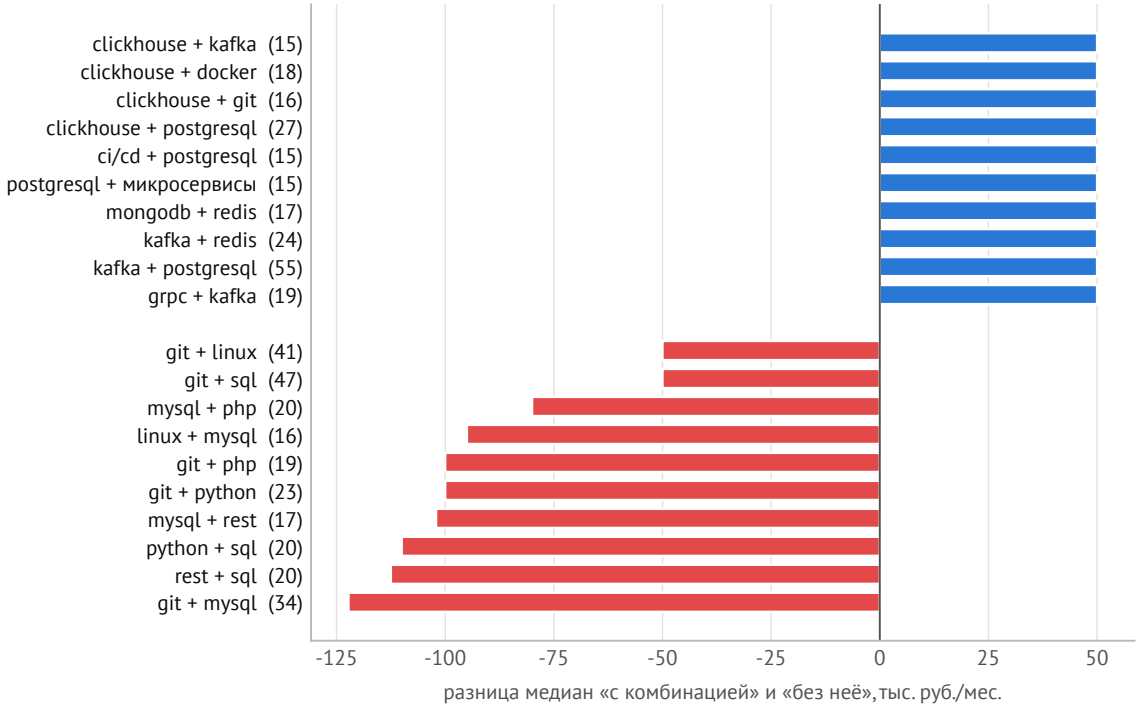


Рис. 1 – Влияние пар навыков на медиану зарплаты; в скобках — число предложений

Таблица 4 – Тройки навыков: повышают и понижают медиану

комбинация	предл.	медиана с	медиана без	Δ, %	p	Δ 3–6 лет
grpc + kafka + postgresql	16	300 000	250 000	+20,0	0,204	14 500
kafka + postgresql + redis	16	300 000	250 000	+20,0	0,204	62 625
kafka + postgresql + rabbitmq	19	300 000	250 000	+20,0	0,247	0
docker + kafka + postgresql	21	300 000	250 000	+20,0	0,225	54 000
grpc + postgresql + rest	20	300 000	250 000	+20,0	0,168	0
docker + grpc + postgresql	16	275 000	250 000	+10,0	0,507	-82 500
git + mysql + postgresql	16	113 100	250 000	-54,8	0,000	—
git + rest + sql	15	140 000	250 000	-44,0	0,005	-66 250
docker + kubernetes + redis	17	200 000	250 000	-20,0	0,272	-50 000
git + postgresql + sql	33	210 975	250 000	-15,6	0,155	-50 000
git + linux + postgresql	24	217 500	250 000	-13,0	0,288	-78 750
docker + git + linux	23	217 500	250 000	-13,0	0,307	-62 500

Ниже всего медиана в связках с MySQL, PHP, Python, Git и REST + SQL: на 40–49 % у пар и на 55 % у тройки «git + mysql + postgresql». Это устойчиво: $p \leq 0{,}02$, и при опыте 3–6 лет разница у этих пар остаётся отрицательной. Связки ClickHouse, Kafka, gRPC и CI/CD с PostgreSQL дают +20 %, но по отдельности это в пределах случайного разброса (p от 0,05 до 0,3); у Kafka и ClickHouse разница положительна и при одинаковом опыте, так что устойчиво только направление. Причинность отсюда не следует: навыки говорят о типе вакансии (уровне, домене, работодателе).

6. Этап 6. Три региона

Выбраны три крупнейших по числу Go-вакансий региона. Зарплаты считаются по уникальным предложениям внутри региона; регион на hh.ru — место размещения вакансии.

Таблица 5 – Сравнение регионов

показатель	Москва	Санкт-Петербург	Екатеринбург	Весь сегмент Go
вакансий Go	1 743	389	76	2 751
доля сегмента, %	63,4	14,1	2,8	100,0
работодателей	464	146	40	677
с зарплатой в рублях, %	20,7	22,4	31,6	22,9
удалённая работа, %	37,3	37,8	43,4	35,8
опыт: нет, %	1,9	1,3	0,0	2,3
опыт: 1–3 года, %	21,6	22,4	48,7	25,4
опыт: 3–6 лет, %	66,8	67,1	48,7	63,8
опыт: больше 6 лет, %	9,7	9,3	2,6	8,5
с навыками, %	73,1	76,9	65,8	74,1
навыков на вакансию с навыками	5,5	5,6	5,2	5,7
предложений с зарплатой	302	71	19	465
зарплата p25, руб.	218 125	217 500	120 000	175 125
медиана, руб.	300 000	270 000	200 000	250 000
зарплата p75, руб.	375 000	395 750	251 150	350 000

Таблица 6 – Разница медиан между регионами (перестановочный тест)

регион A	регион B	предл. A	предл. B	разница медиан, руб.	p
Москва	Санкт-Петербург	302	71	30 000	0,152
Москва	Екатеринбург	302	19	100 000	0,005
Санкт-Петербург	Екатеринбург	71	19	70 000	0,047

Таблица 7 – Доля вакансий с навыком среди вакансий региона с навыками, %

навык	Москва	Санкт-Петербург	Екатеринбург	весь сегмент
postgresql	47,6	50,2	36,0	48,6
docker	29,4	29,8	22,0	28,4
kafka	26,3	24,7	10,0	25,1
kubernetes	23,2	23,1	0,0	20,8
redis	16,3	16,7	14,0	15,6
rabbitmq	14,3	13,7	18,0	15,3
linux	18,3	19,7	30,0	20,0
git	19,2	13,4	22,0	20,8
sql	18,8	22,4	22,0	21,2
python	8,9	8,7	20,0	10,2
php	7,5	6,4	12,0	7,9
clickhouse	8,3	13,0	8,0	9,0

Москва даёт 63,4 % вакансий сегмента, медиана 300 000, две трети вакансий требуют опыта 3–6 лет. Санкт-Петербург по спросу почти не отличается от Москвы; его медиана ниже на 30 тыс., но при 71 предложении это статистически не подтверждается ($p = 0{,}152$). Екатеринбург — небольшой и заметно другой рынок: половина вакансий на опыт 1–3 года, больше всего удалённых (43,4 %), медиана 200 000 ниже московской ($p = 0{,}005$) и петербургской ($p = 0{,}047$). Kafka там указана у 10 % вакансий против 25 % по сегменту, Kubernetes нет ни в одной, чаще встречаются Linux и Python. Предложений с зарплатой в Екатеринбурге 19, так что его цифры ориентировочные.

7. Профилирование

Замеры сделаны на Apple M3 Pro (12 ядер, 18 ГиБ), macOS 27.0.1, Docker Desktop (ВМ: 12 vCPU, 7,75 ГиБ), Python 3.13, DuckDB 1.5.6. Класс `hw/profiling.py:Profile` меряет время через `perf_counter`, процессорное время через `getrusage` процесса и потомков, а пик памяти считает фоновый поток, раз в 20 мс суммируя RSS всех процессов. Память указана в МиБ (2^{20} байт). `scripts/benchmark.py` запускает каждую конфигурацию отдельным процессом по 3 раза после прогрева; у всех конфигураций одного способа совпадает отпечаток результата, то есть оптимизации ответ не меняют.

Основные конфигурации на исходном CSV: MapReduce на 8 процессах — 4,10 с и 1 209 МиБ, MapReduce в одном процессе — 4,58 с и 156 МиБ, DuckDB — 4,03 с и 360 МиБ.

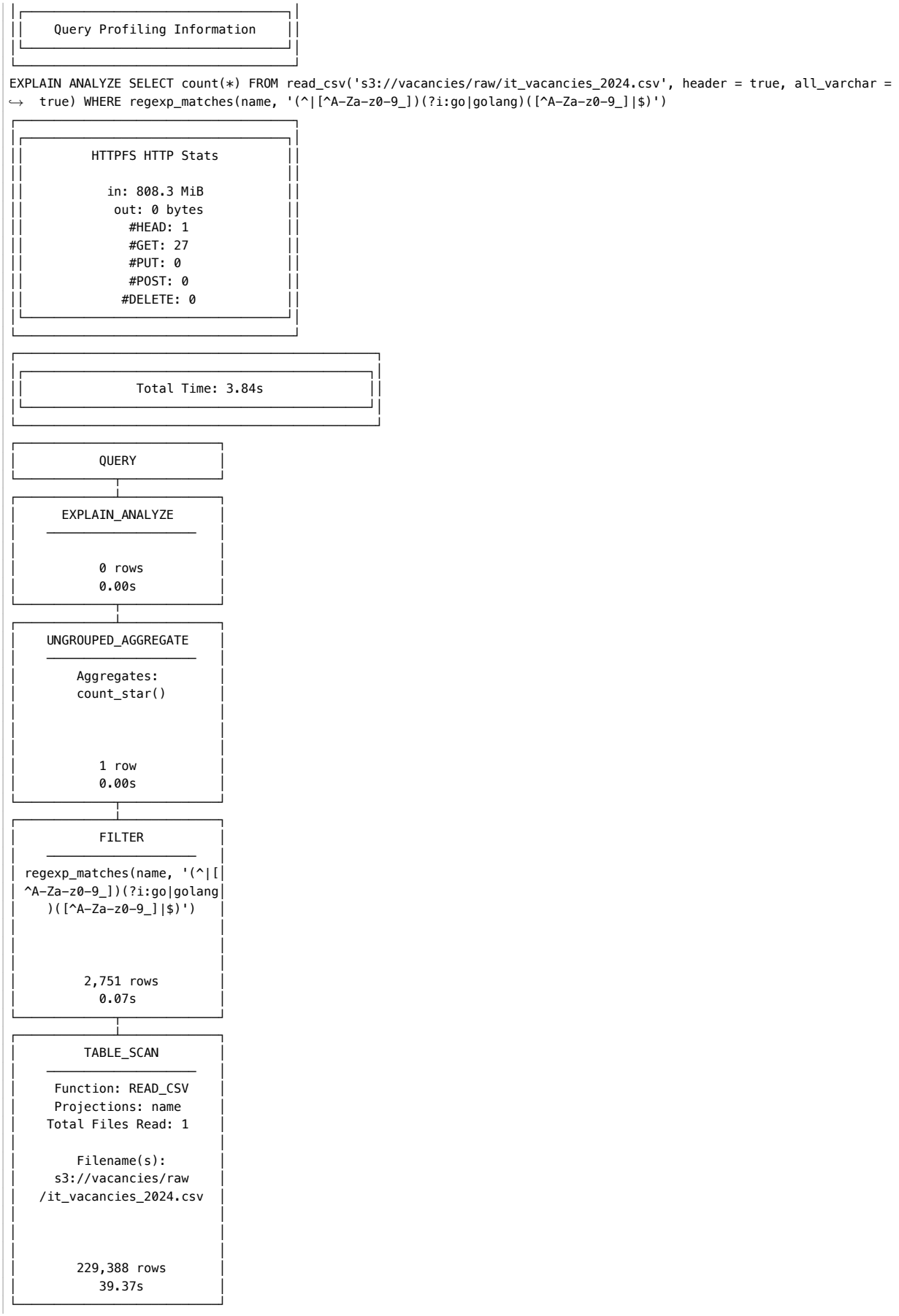
Внутри map-таска (cProfile одного сплита, `scripts/profile_map_task.py`) с первой версией предфильтра 67 % времени уходило на поиск регулярного выражения, с итоговой — 70 % на чтение из сокета:

```
предфильтр: regex; сплит 5: байты [167,772,160, 201,326,592)
57508 function calls (57452 primitive calls) in 0.568 seconds
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      599    0.380    0.001    0.380    0.001 {method 'search' of 're.Pattern' objects}
     2076    0.154    0.000    0.154    0.000 {method 'recv_into' of '_socket.socket' objects}
        1    0.010    0.010    0.010    0.010 {method 'count' of 'bytearray' objects}
```

```
предфильтр: fast; сплит 5: байты [167,772,160, 201,326,592)
58257 function calls (58201 primitive calls) in 0.176 seconds
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
     2039    0.124    0.000    0.124    0.000 {method 'recv_into' of '_socket.socket' objects}
        1    0.010    0.010    0.010    0.010 {method 'count' of 'bytearray' objects}
        1    0.007    0.007    0.007    0.007 {method 'lower' of 'bytearray' objects}
```

У DuckDB по `EXPLAIN ANALYZE` почти всё время запроса (3,84 с) занимает чтение CSV из RustFS, а тройки считаются за 0,0140 с (план троек шире страницы и лежит в `logs/04_duckdb.txt`):

```
-- EXPLAIN ANALYZE: сканирование объекта и фильтр Go
┌─────────────────────────────────────┐
```



Простое чтение объекта без обработки (`scripts/bench_storage.py`) упирается в 240 МиБ/с, то есть в 3,4 с на весь файл, уже при 2 соединениях; RustFS при этом загружен меньше чем на ядро. Вероятное узкое место — сеть Docker Desktop между macOS и ВМ.

```
Объект 847,619,384 байт, 26 диапазонов по 32 МиБ, повторов 2
 потоков  время, с   МиБ/с  CPU RustFS, % (ср./макс.)
       1      3.69     219                42 / 58.58
       2      3.37     240                47 / 53.17
       4      3.40     238                52 / 66.15
       8      3.38     239                44 / 77.59
      16      3.88     208               120 / 464.97
```

8. Бонус: оптимизация

Таблица 8 – MapReduce: подсчёт по CSV 847,6 МБ, оптимизации включались нарастающим итогом

конфигурация	время, с [мин–макс]	CPU, с	RSS, МиБ	быстрее	память
MR «до»: 1 процесс, разбор всех строк, без combiner, shuffle сортировкой	8,33 [8,17–8,77]	5,89	1 011	1,0×	1,0×
MR: 1 процесс, построчное декодирование, hash-shuffle в файлы; без предфильтра и combiner	7,97 [7,88–8,05]	5,85	184	1,0×	5,5×
+ combiner на стороне map	7,83 [7,79–7,88]	5,81	183	1,1×	5,5×
+ предфильтр строк, первая версия (регулярка с lookbehind)	15,69 [15,50–15,78]	13,39	135	0,5×	7,5×
+ предфильтр строк, итоговая версия (поиск по литералу)	4,58 [4,52–4,75]	2,62	156	1,8×	6,5×
8 процессов, первая версия предфильтра	5,11 [4,98–5,22]	18,81	1 250	1,6×	0,8×
+ 2 процесса	4,21 [4,20–4,28]	3,76	381	2,0×	2,7×
+ 4 процесса	3,84 [3,79–3,91]	4,89	655	2,2×	1,5×
+ 8 процессов (как в запуске этапа 3)	4,10 [4,09–4,10]	8,20	1 209	2,0×	0,8×
+ 12 процессов	4,46 [4,40–4,50]	12,02	1 533	1,9×	0,7×
8 процессов, сплит 8 МиБ	4,12 [4,02–4,30]	8,51	808	2,0×	1,3×
8 процессов, сплит 64 МиБ	4,38 [4,30–4,44]	8,50	1 522	1,9×	0,7×
8 процессов, сплит 128 МиБ	4,47 [4,46–4,56]	7,58	1 812	1,9×	0,6×
MR по выделенному сегменту Go, 1 процесс	0,28 [0,28–0,30]	0,24	93	29,4×	10,9×

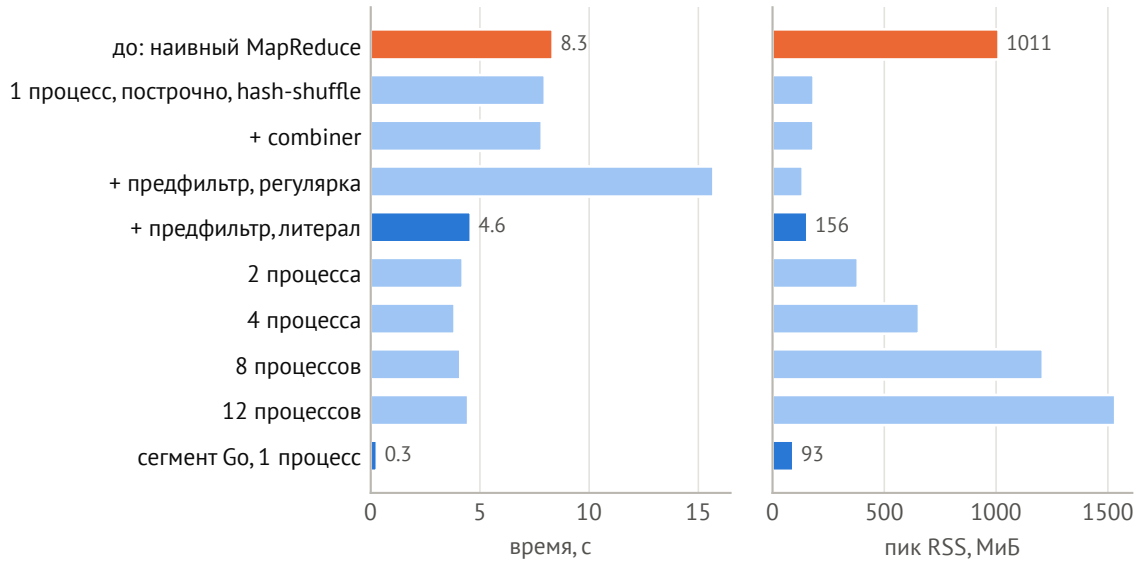


Рис. 2 – MapReduce: время и пик памяти (оранжевый — «до», синий — итоговые варианты)

«До» — наивный MapReduce: один процесс, каждый чанк целиком декодируется в строку и разбирается через `StringIO`, все пары копятся в один список, shuffle — сортировка. Что

изменилось дальше:

1. Построчное декодирование и hash-shuffle в файлы снизили пик памяти в 5,5 раза. По `tracemalloc` куча Python у наивного варианта не больше 238 МиБ, а RSS растёт почти до 1 ГиБ: аллокатор macOS не сразу возвращает освобождённые крупные блоки:

```
сплит  RSS, МБ  куча Python сейчас, МБ  пик кучи, МБ  пар (key, value)
    0      214                      37           146            10,116
    5      509                      40           150            36,003
   10      645                      44           155            63,856
   15      828                      48           238            94,592
   20      931                      52           238           129,845
   25      796                      28           238           161,469
ru_maxrss: 987 МБ
```

2. Combiner сократил shuffle со 161 469 пар до 105 324 ключей.
3. Предфильтр по слову go/golang оставляет к разбору 3,8 % строк. Первая версия на регулярном выражении с lookbehind и `re.I` оказалась медленнее, чем вообще без предфильтра (15,69 против 7,83 с): такой шаблон отключает быструю проверку литерального префикса. Итоговая версия ищет литерал `go` по копии чанка в нижнем регистре и проверяет границу слова вручную — 4,58 с, в 3,4 раза быстрее. Сплит читается сразу в заранее выделенный `bytearray`, без промежуточных копий.
4. С медленным предфильтром 8 процессов ускоряли в 3,1 раза. С быстрым один процесс почти успевает за хранилищем: 2–12 процессов дают 3,84–4,46 с против 4,58 с, а память растёт примерно на 125 МиБ на процесс.
5. Выделенный один раз сегмент Go (1,3 % исходного объёма) тот же MapReduce считает за 0,28 с.

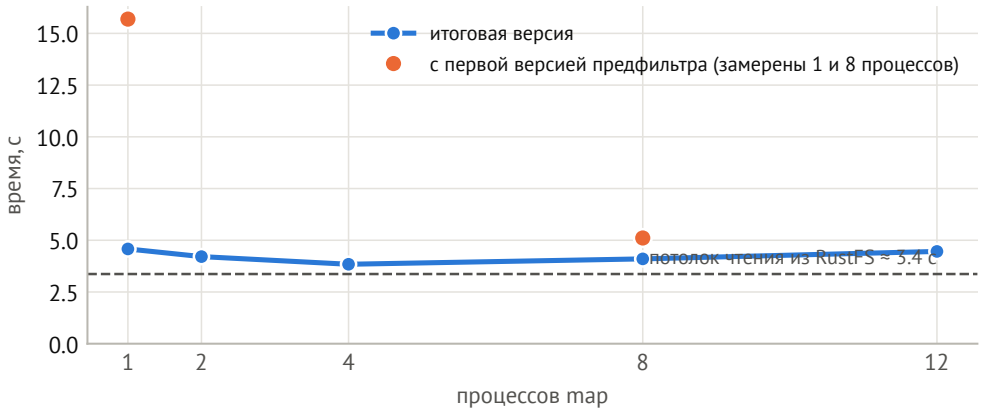


Рис. 3 – Время MapReduce в зависимости от числа процессов; пунктир — время простого чтения объекта на максимальной скорости RustFS

В одном процессе MapReduce ускорен в 1,8 раза (8,33 → 4,58 с), пик памяти снижен в 6,5 раза (1 011 → 156 МиБ).

Таблица 9 – DuckDB: подсчёт навыков, пар и троек

конфигурация	время, с [мин–макс]	CPU, с	RSS, МиБ	быстрее	память
DuckDB «до»: 3 запроса, каждый читает CSV с автоопределением	12,11 [11,61–14,37]	8,20	536	1,0×	1,0×
DuckDB: одно чтение CSV в таблицу, 1 поток	4,71 [4,10–4,74]	2,69	270	2,6×	2,0×
DuckDB: одно чтение CSV в таблицу, все потоки	4,03 [3,86–4,19]	2,75	360	3,0×	1,5×
DuckDB: Parquet, 13 столбцов, zstd	0,15 [0,15–0,15]	0,23	182	78,7×	2,9×
DuckDB: выделенный сегмент Go (CSV)	0,32 [0,31–0,32]	0,37	167	37,9×	3,2×

У DuckDB наивный вариант (`sql/naive_counts.sql`) трижды читает CSV с автоопределением схемы. Одно чтение в таблицу ускоряет в 3,0 раза. Parquet с 13 нужными столбцами (1,0 % исходного объёма, конвертация 5,3 с, `scripts/make_curated.py`) даёт 0,15 с, примерно в 79 раз быстрее «до».

```
Исходный объект s3://vacancies/raw/it_vacancies_2024.csv: 847,619,384 байт
s3://vacancies/curated/vacancies.parquet: 8,170,694 байт (1.0 % исходного), записан за 5.3 с
s3://vacancies/segments/go/it_vacancies_go.csv: 10,699,361 байт (1.3 % исходного), записан за 4.5 с
Строк: Parquet 229,388, сегмент Go 2,751
```

Полный вывод замеров (`scripts.benchmark`), по которому построены таблицы:

```
Прогрев...
Повторов на конфигурацию: 3; время — медиана [мин–макс], память — максимум пика RSS

== mapreduce
mr_baseline     8.33 с  [8.17–8.77]  CPU   5.89 с  пик RSS  1011.3 МБ  digest 4020496dcffd0b57  — MR «до»: 1 процесс,
↪  разбор всех строк, без combiner, shuffle сортировкой
mr_w1_plain     7.97 с  [7.88–8.05]  CPU   5.85 с  пик RSS   183.9 МБ  digest 4020496dcffd0b57  — MR: 1 процесс,
↪  построчное декодирование, hash-shuffle в файлы; без предфильтра и combiner
mr_w1_comb      7.83 с  [7.79–7.88]  CPU   5.81 с  пик RSS   183.3 МБ  digest 4020496dcffd0b57  — + combiner на стороне map
mr_w1_regex    15.69 с  [15.50–15.78]  CPU  13.39 с  пик RSS   135.0 МБ  digest 4020496dcffd0b57  — + предфильтр строк,
↪  первая версия (регулярка с lookbehind)
mr_w1_pref      4.58 с  [4.52–4.75]  CPU   2.62 с  пик RSS   155.6 МБ  digest 4020496dcffd0b57  — + предфильтр строк,
↪  итоговая версия (поиск по литералу)
mr_w8_regex     5.11 с  [4.98–5.22]  CPU  18.81 с  пик RSS  1249.5 МБ  digest 4020496dcffd0b57  — 8 процессов, первая
↪  версия предфильтра
mr_w2           4.21 с  [4.20–4.28]  CPU   3.76 с  пик RSS   381.4 МБ  digest 4020496dcffd0b57  — + 2 процесса
mr_w4           3.84 с  [3.79–3.91]  CPU   4.89 с  пик RSS   654.9 МБ  digest 4020496dcffd0b57  — + 4 процесса
mr_w8           4.10 с  [4.09–4.10]  CPU   8.20 с  пик RSS  1209.0 МБ  digest 4020496dcffd0b57  — + 8 процессов (как в
↪  запуске этапа 3)
mr_w12          4.46 с  [4.40–4.50]  CPU  12.02 с  пик RSS  1532.8 МБ  digest 4020496dcffd0b57  — + 12 процессов
mr_w8_s8        4.12 с  [4.02–4.30]  CPU   8.51 с  пик RSS   808.5 МБ  digest 4020496dcffd0b57  — 8 процессов, сплит 8 МиБ
mr_w8_s64       4.38 с  [4.30–4.44]  CPU   8.50 с  пик RSS  1522.5 МБ  digest 4020496dcffd0b57  — 8 процессов, сплит 64 МиБ
mr_w8_s128      4.47 с  [4.46–4.56]  CPU   7.58 с  пик RSS  1811.5 МБ  digest 4020496dcffd0b57  — 8 процессов, сплит 128
↪  МиБ
mr_segment      0.28 с  [0.28–0.30]  CPU   0.24 с  пик RSS    93.0 МБ  digest 4020496dcffd0b57  — MR по выделенному
↪  сегменту Go, 1 процесс

== duckdb
db_naive       12.11 с  [11.61–14.37]  CPU   8.20 с  пик RSS   536.4 МБ  digest 00075e61ca94fa50  — DuckDB «до»: 3
↪  запроса, каждый читает CSV с автоопределением
db_csv_t1       4.71 с  [4.10–4.74]  CPU   2.69 с  пик RSS   269.8 МБ  digest 00075e61ca94fa50  — DuckDB: одно чтение CSV
↪  в таблицу, 1 поток
db_csv          4.03 с  [3.86–4.19]  CPU   2.75 с  пик RSS   360.5 МБ  digest 00075e61ca94fa50  — DuckDB: одно чтение CSV
↪  в таблицу, все потоки
db_parquet      0.15 с  [0.15–0.15]  CPU   0.23 с  пик RSS   182.0 МБ  digest 00075e61ca94fa50  — DuckDB: Parquet, 13
↪  столбцов, zstd
db_segment      0.32 с  [0.31–0.32]  CPU   0.37 с  пик RSS   167.2 МБ  digest 00075e61ca94fa50  — DuckDB: выделенный
↪  сегмент Go (CSV)
duckdb: результат одинаковый во всех конфигурациях (00075e61ca94fa50)
mapreduce: результат одинаковый во всех конфигурациях (4020496dcffd0b57)
```

9. Выводы

1. RustFS в Docker Compose (4 папки-диска, EC 2+2) хранит датасет и все результаты; целостность подтверждена SHA-256.
2. Свой MapReduce режет объект на сплиты без потери и дублирования строк и считает частоты навыков, пар и троек так же, как DuckDB: совпадение полное, включая медианы зарплат.
3. В Go-вакансиях чаще всего вместе с Go указаны PostgreSQL и Docker. Устойчиво ниже платят в вакансиях со стеком «Git + MySQL + PHP/Python + REST»; связки с ClickHouse, Kafka и gRPC стоят выше медианы, но на этой выборке разница не значима.
4. Москва и Петербург близки и по навыкам, и по зарплатам; Екатеринбург младше по опыту, чаще удалённый, с другим набором навыков и более низкой медианой.
5. После оптимизации время обоих способов упирается в чтение из хранилища, и дальше помогает только раскладка данных: Parquet или выделенный сегмент.